

Nivel 1

Nivel 1

Descripción

Como Analytics Engineer, tu responsabilidad no es sólo entregar datos, sino asegurar la escalabilidad y eficiencia de la plataforma.

En este sprint final, rediseñarás la arquitectura de los datos para optimizar costes (**Partitioning**), introducirás lógica analítica avanzada (**Window Functions**) y modelarás estructuras complejas (**Arrays**) para cerrar el ciclo con la automatización.



Importante

- » **Control de Costes** : En BigQuery, cada consulta cuesta dinero. Antes de ejecutar nada, fíjate siempre en el mensaje del validador ("Dry Run") arriba a la derecha.
- » **Ubicación de los Datos** : Asegúrate de que tu Dataset esté en la región EU (Multiple regiones) o las conexiones fallarán.

Limitaciones del Sandbox

La zona de pruebas de BigQuery (Sandbox) está sujeta a los siguientes límites:

- » Se aplican todos los límites y cuotas habituales de BigQuery.
- » Tienes 10 GB de almacenamiento activo y 1 TB de datos de consulta procesados cada mes sin coste alguno.
- »  **Caducidad** : Todas las tablas, vistas y particiones caducan de forma automática después de **60 días** .
- » No admite funciones avanzadas como: Transmisión de datos (Streaming), DML (`UPDATE` / `DELETE` masivos) o el Servicio de transferencia de datos.

Nivel 1: Entorno e Ingesta Híbrida (Code-First)

Escenario:

El salpicadero tarda demasiado en cargar y el coste es inaceptablemente alto. Se ha identificado que el problema crítico son los informes que creen ventas con regiones geográficas.

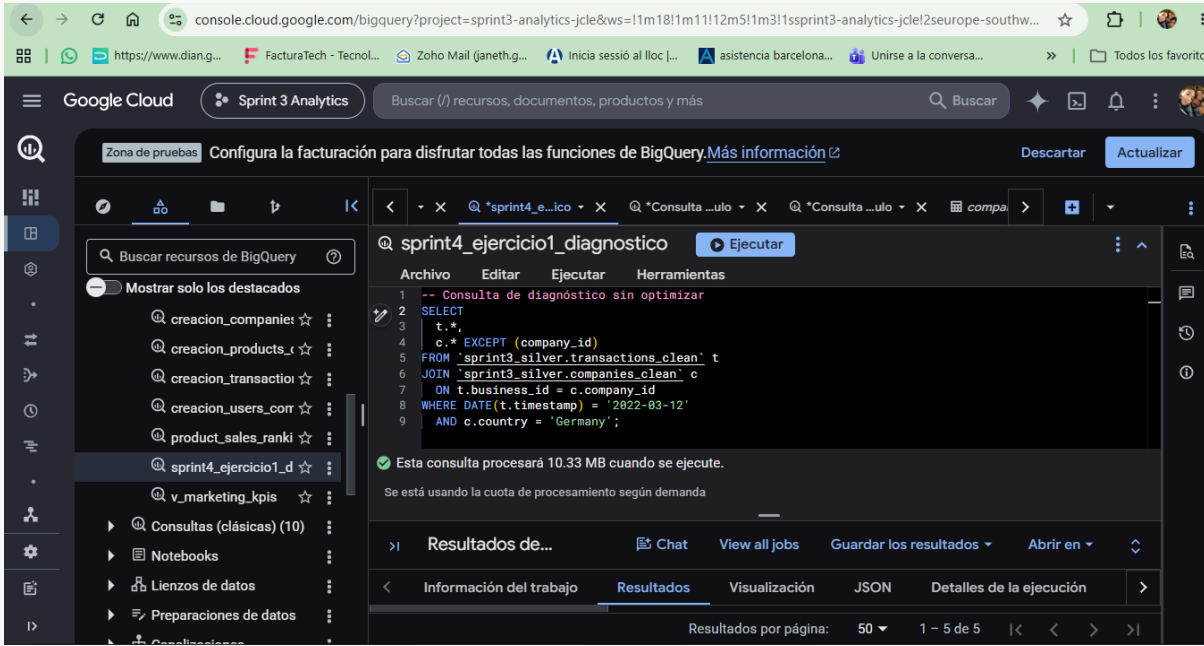
Ejercicio 1: Consulta sobre Tabla no Optimizada (Diagnóstico)

El Country Manager de **Alemania** necesita revisar urgentemente las transacciones del día **12 de marzo de 2022**.

Tarea:

- » 1. Escribe la consulta que une (JOIN) transacciones y compañías.
 - » 2. Filtra los resultados por la fecha indicada y el país "Germany".
 - » 3. Sin ejecutar la consulta, realiza un "Dry Run" (auditoría de costes).
- 👁 Observación: Fíjate en que BigQuery lee casi toda la mesa a pesar de pedir sólo un día (Full Table Scan).*

Resultado:



The screenshot shows the Google Cloud BigQuery console interface. The main editor displays a SQL query titled "sprint4_ejercicio1_diagnostico". The query is as follows:

```
-- Consulta de diagnóstico sin optimizar
SELECT
  t.*,
  c.* EXCEPT (company_id)
FROM `sprint3_silver.transactions_clean` t
JOIN `sprint3_silver.companies_clean` c
  ON t.business_id = c.company_id
WHERE DATE(t.timestamp) = '2022-03-12'
  AND c.country = 'Germany';
```

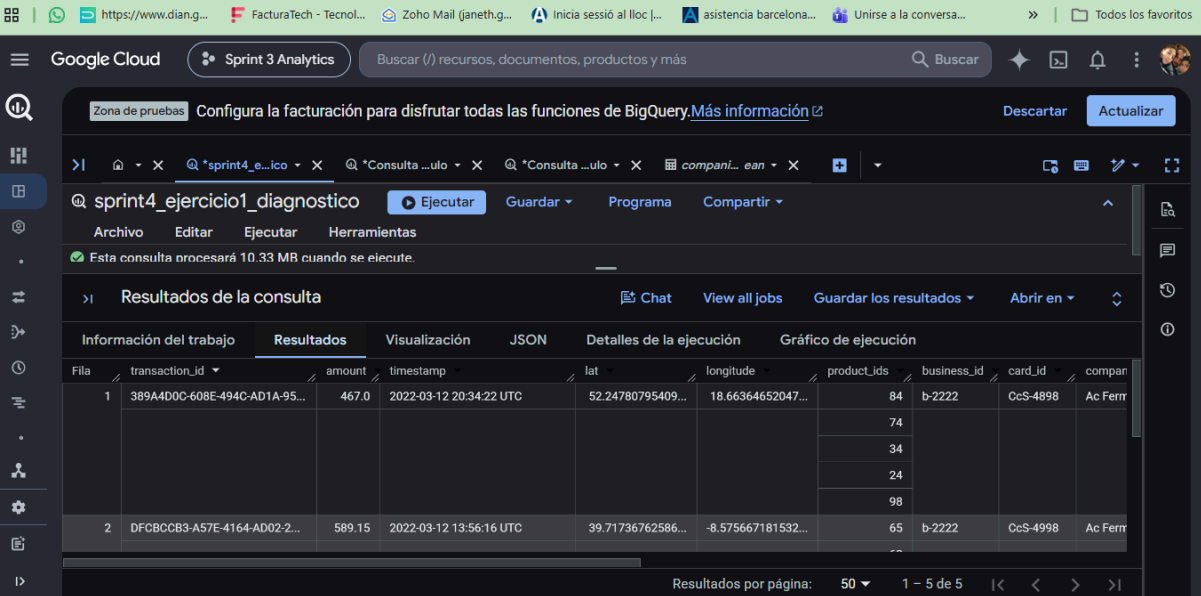
Below the query editor, a green checkmark icon indicates that the query will process 10.33 MB when executed. A message states: "Esta consulta procesará 10.33 MB cuando se ejecute. Se está usando la cuota de procesamiento según demanda".

The interface also shows a sidebar with a list of resources, including "sprint4_ejercicio1_d", and a bottom section with tabs for "Resultados de...", "Chat", "View all jobs", "Guardar los resultados", and "Abrir en".

link código SQL consulta compartida:

<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest!13s69781b27-5697-47b6-a5c6-117fd77897a6!2e1>

Al ejecutar la consulta devuelve 5 registros para el mismo día y país.



The screenshot shows the Google Cloud BigQuery interface. At the top, there's a navigation bar with 'Google Cloud' and 'Sprint 3 Analytics'. Below it, a search bar and a 'Buscar' button. The main area displays the query 'sprint4_ejercicio1_diagnostico' with an 'Ejecutar' button. Below the query, there's a status message: 'Esta consulta procesará 10.33 MB cuando se ejecute.' The results section shows a table with columns: 'Fila', 'transaction_id', 'amount', 'timestamp', 'lat', 'longitud', 'product_ids', 'business_id', 'card_id', and 'company'. The table contains 5 rows of data. The first row has a transaction_id starting with '389A4D0C-608E-494C-AD1A-95...', an amount of 467.0, a timestamp of 2022-03-12 20:34:22 UTC, and a company of 'Ac Ferr'. The second row has a transaction_id starting with 'DFCBB3-A57E-4164-AD02-2...', an amount of 589.15, a timestamp of 2022-03-12 13:56:16 UTC, and a company of 'Ac Ferr'. The table is paginated, showing 1-5 of 5 results.

Fila	transaction_id	amount	timestamp	lat	longitud	product_ids	business_id	card_id	company
1	389A4D0C-608E-494C-AD1A-95...	467.0	2022-03-12 20:34:22 UTC	52.24780795409...	18.66364652047...	84	b-2222	CcS-4898	Ac Ferr
						74			
						34			
						24			
						98			
2	DFCBB3-A57E-4164-AD02-2...	589.15	2022-03-12 13:56:16 UTC	39.71736762586...	-8.575667181532...	65	b-2222	CcS-4998	Ac Ferr

Ejercicio 2: Re-arquitectura y Optimización del Almacenamiento (Partition & Cluster)

Escenario:

Tras el diagnóstico del ejercicio anterior, hemos confirmado que filtrar la tabla de transacciones por fecha o por empresa provoca un *Full Table Scan* (lectura completa de la tabla), lo que es ineficiente. Para solucionarlo en la capa de consumo (*Gold*), materializaremos una versión optimizada de la tabla de hechos.

Problema del sandbox: al utilizar un entorno gratuito sin tarjeta de crédito, BigQuery borra automáticamente los datos de las particiones con más de 60 días de antigüedad. Dado que nuestros datos históricos son antiguos, si particionamos directamente la tabla, ésta acabará quedando vacía. Para evitarlo, lo resolveremos en dos pasos: primero, actualizaremos automáticamente las fechas de los datos y después aplicaremos la optimización física.

Tarea:

Paso 1: Generación de Datos Recientes (Mocking Data)

Crea una tabla intermedia llamada `sprint3_silver.transactions_recent` a partir de la tabla `sprint3_silver.transactions_clean`. Tu objetivo es mantener todas las columnas, pero sustituir el `timestamp` original

por uno nuevo, generado aleatoriamente para que caiga dentro de los últimos 50 días .

» **Pista Técnica :** Para hacerlo sin escribir todas las columnas a mano, puedes utilizar `SELECT * EXCEPT(timestamp)`. Para calcular la fecha aleatoria, deberás restar una cantidad de días al instante actual. Te serán muy útiles las funciones `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()`, y la combinación `CAST(RAND() * 50 AS INT64)` para generarlos días de desfase.

Paso 2: Creación de la Mesa Optimizada (Partitioning & Clustering)

Ahora, crea tu tabla definitiva `sprint3_ gold.fact_transactions_optimized` a partir de los datos recientes que acabas de generar en `transactions_recent` . Tienes que construir la sentencia DDL para configurar la tabla con las siguientes estrategias físicas de almacenamiento :

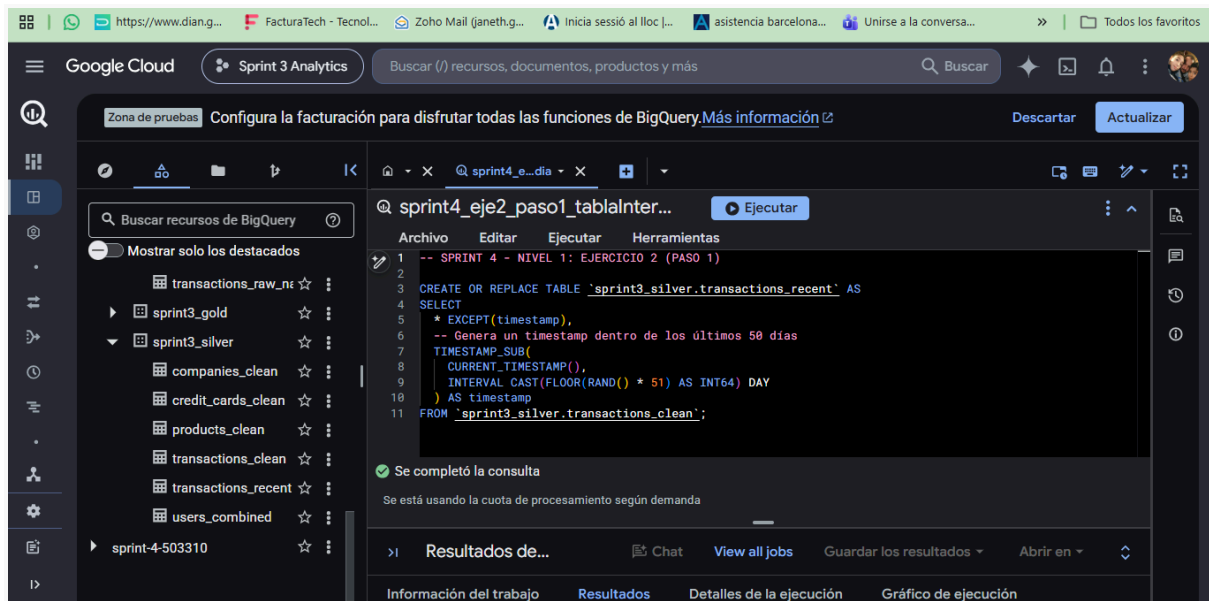
» **Particionamiento (Partitioning)** : Divide la tabla por la fecha del campo `DATE(timestamp)`. Esto permitirá que las consultas que filtren por día (`WHERE date = ...`) sólo lean la partición necesaria.

» **Clustering** : Ordena los datos dentro de cada partición por `business_id`. Esto acelerará drásticamente los filtros por compañía y los cruces (`JOINS`) con la dimensión de empresas.

👉 Entrega:

- » El código SQL utilizado para el Paso 1.
- » El código DDL (`CREATE OR REPLACE TABLE...`) utilizado en el Paso 2.
- » Una captura de pantalla de los " Detalles " de la nueva tabla `fact_transactions_optimized` donde se pueda comprobar que está particionada y agrupada en clústeres correctamente .

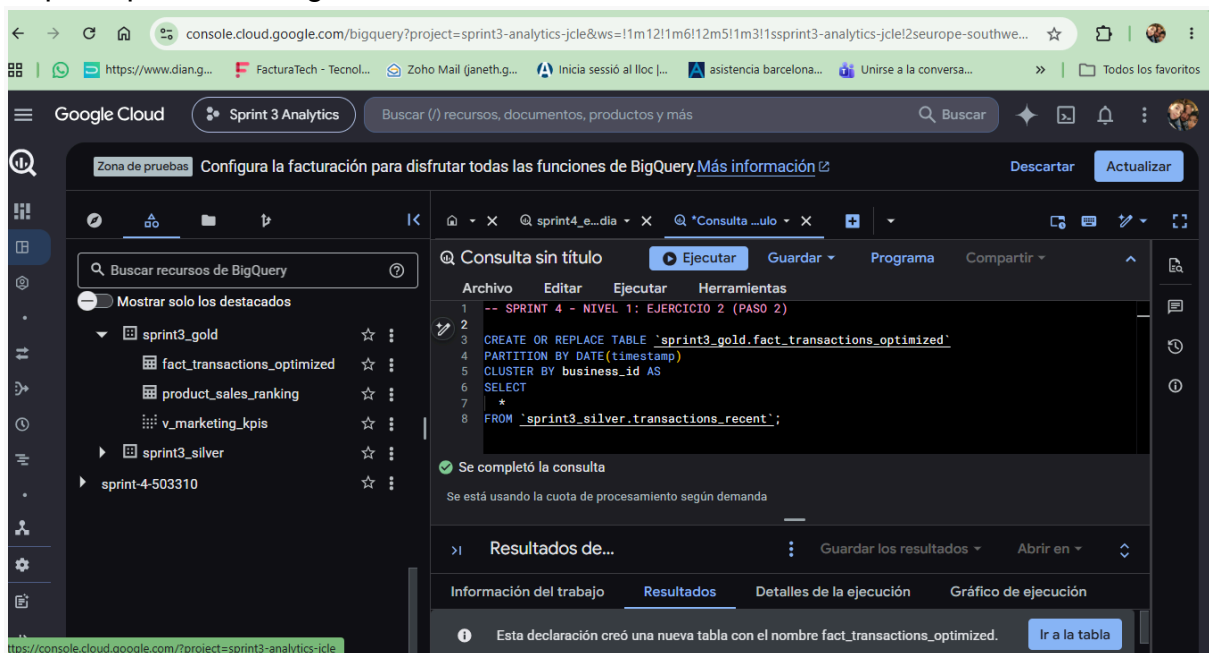
Captura pantalla código Utilizado Paso 1:



Link código consulta paso 1:

<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest!13s375428fc-7806-4b7c-976a-44f16227b7ff!2e1>

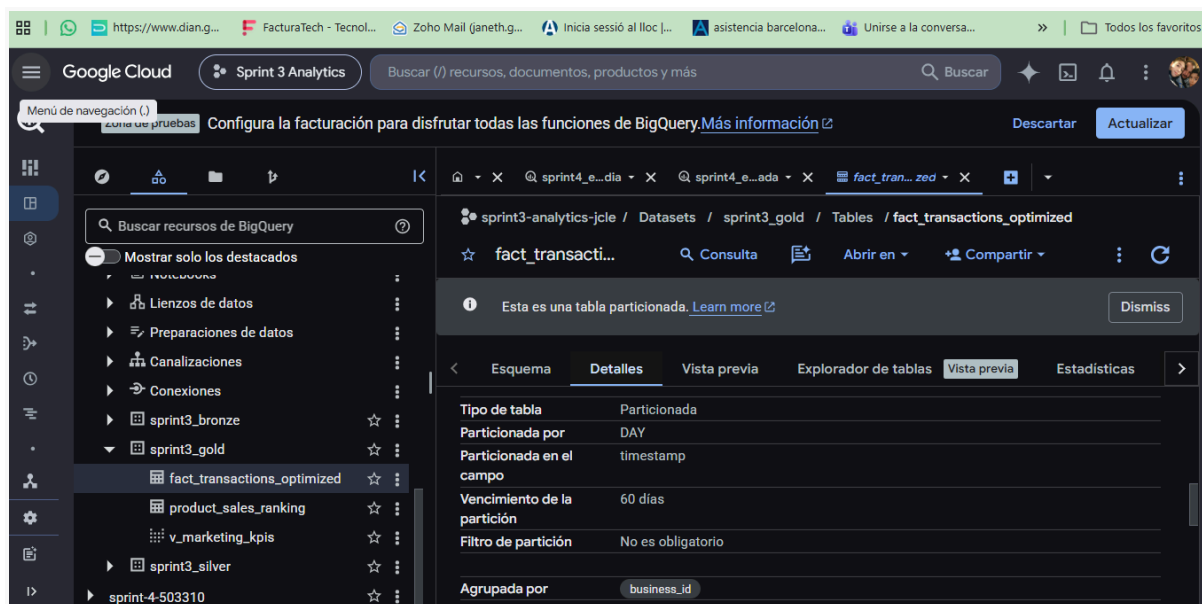
Captura pantalla código Utilizado Paso 2



Link código consulta paso 2:

<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest!13s5cdc5ef3-e141-416f-9585-b3bac42f4005!2e1>

Captura Nueva Tabla con los detalles:



Ejercicio 3: La Prueba del Algodón (Benchmark)

Escenario:

Escenario : Ya has creado la mesa `sprint3_gold.fact_transactions_optimized` con las estrategias de Particionamiento y Clustering. Ahora toca demostrar al equipo la mejora de rendimiento y el ahorro de costes que has conseguido . Simularemos una petición típica de negocio : " Queremos ver todas las transacciones de los últimos 30 días " .

Tarea:

El objetivo es claro y directo: **aplicar exactamente la misma consulta a dos tablas diferentes y comparar su coste computacional** .

Construye una consulta SQL que seleccione todas las columnas (`SELECT *`) y filtre los datos de los últimos 30 días.

⚠ NO ejecutes las consultas todavía. En este ejercicio sólo utilizaremos el validador de BigQuery ("Dry Run", arriba a la derecha del editor) para visualizar los **Bytes procesados** de forma gratuita.

Haz la comparativa siguiendo estos dos pasos:

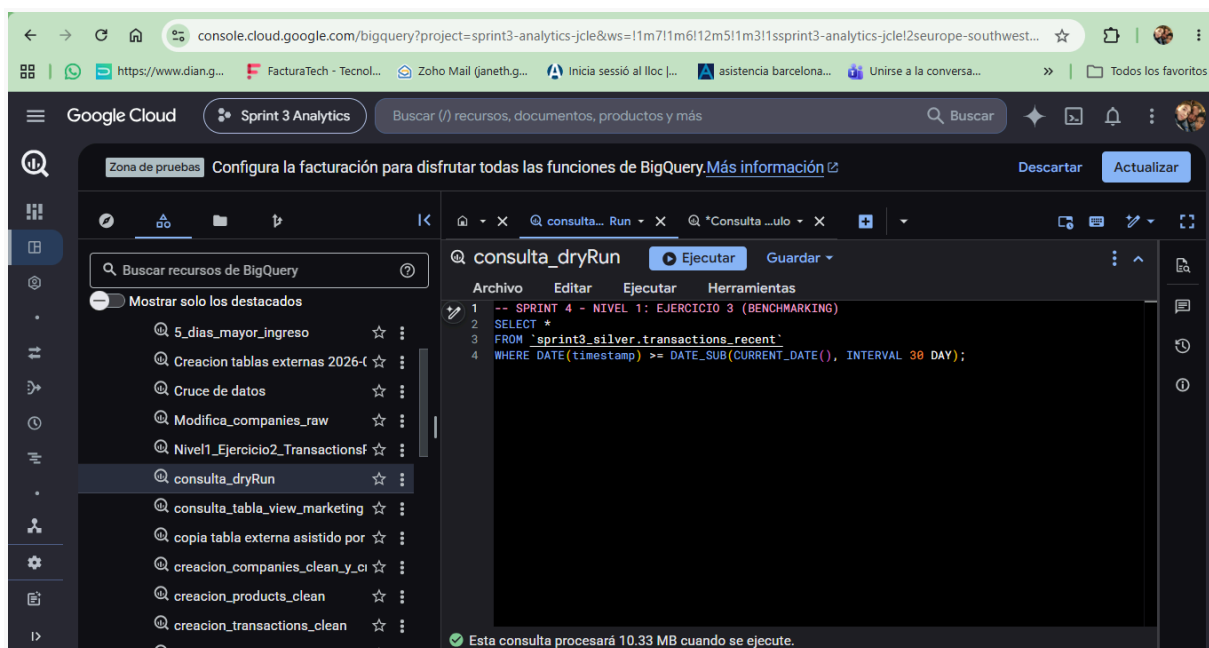
- » **Paso 1 (Tabla no optimizada):** Apunta tu consulta en la tabla `sprint3_silver.transactions_recent`. Observa el validador y realiza una captura de pantalla de los bytes que procesaría.
- » **Paso 2 (Tabla optimizada):** Sin cambiar nada de la lógica de tu consulta, modifica sólo el `FROM` porque ahora apunte a la tabla

particionada `sprint3_gold.fact_transactions_optimized`. Observa cómo cambia el validador y realiza la segunda captura.

👉 Entrega:

- » El código SQL base que has diseñado.
- » Las dos capturas de pantalla donde se vea la diferencia de "Bytes procesados" en el validador.
- » Un breve texto calculando cuál es el **% de ahorro** que has conseguido (compara los bytes del Paso 1 respecto a los del Paso 2).

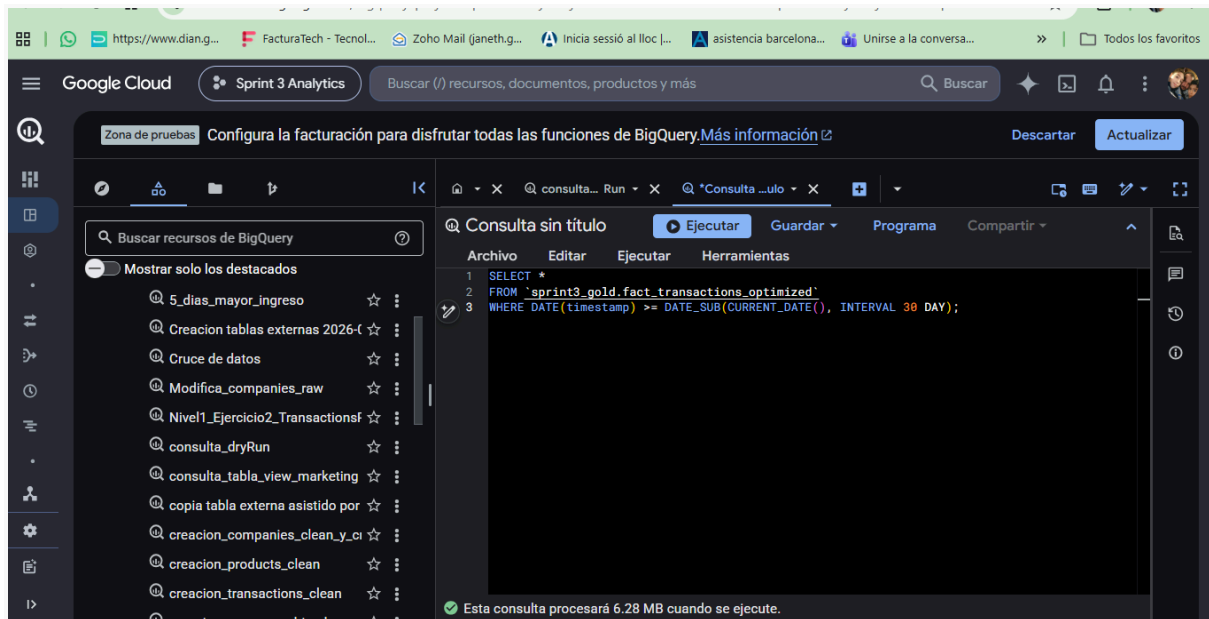
Captura código Consulta Tabla no optimizada y tamaño del Dry Run:



link código consulta no optimizada:

<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest!13sd673bc06-87f0-4765-ba8e-f2bb6a1b5627!2e1>

Captura código consulta optimizada y tamaño del Dry Run:



link código consulta no optimizada:

<https://console.cloud.google.com/bigquery?ws=!1m7!!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest!3sacea575d-8272-456c-b3c7-419ff13f2780!2e1>

De acuerdo a los datos arrojados por cada código de las consultas SQL en el Dry Run podemos apreciar lo siguiente y realizando una comparativa de los Bytes Procesados (Dry Run) tenemos lo siguiente:

Tabla No Optimizada (sprint3_silver.transactions_recent): 10.33 MB

Tabla Optimizada (sprint3_gold.fact_transactions_optimized): 6.28 MB

Porcentaje de Ahorro Conforme: 39.21%

Conclusión:

La implementación de particionamiento diario (PARTITION BY DATE(timestamp)) en la capa Gold permite a BigQuery aplicar la técnica de Partition Pruning, ignorando las particiones fuera del rango de los últimos 30 días. Esto elimina el impacto del Full Table Scan, logrando reducir el procesamiento de datos y los costes cloud en un 39.21% para esta consulta.

Ejercicio 4: Smart Caching (Vistas Materializadas)

Escenario:

El Director General tiene un cuadro de mando que muestra las "**Ventas Totales por Día**". Él (y otros 50 managers) refrescan este gráfico constantemente.

Problema :

Si utilizas una consulta normal o una Vista estándar, cada vez que alguien refresca la página, BigQuery recalcula la suma de millones de filas. Es lento y estás pagando por el mismo cálculo una y otra vez.

Solución :

Utilizar una **Vista Materializada** . Esta tecnología guarda el *resultado* del cálculo en la memoria caché y sólo la actualiza si entran nuevos datos (incremental).

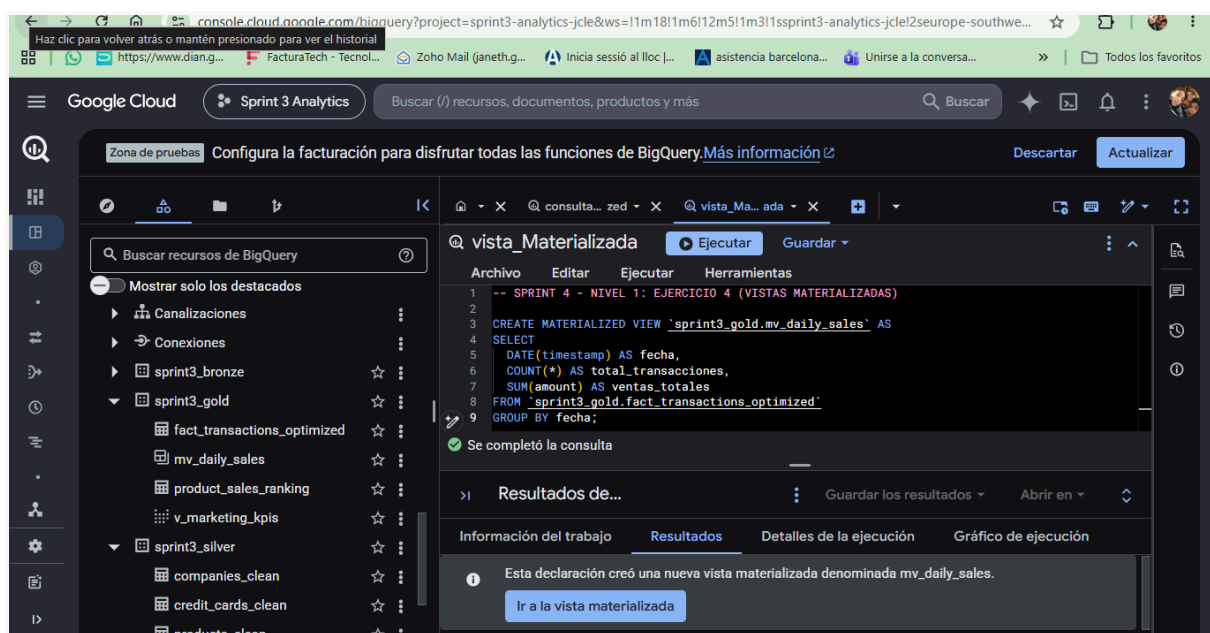
✚ Tarea:

- » Crea una **Vista Materializada** llamada `sprint3_gold.mv_daily_sales` que muestre las **Ventas Totales por Día**.

👉 Entrega:

- » Código para Crear la **Vista Materializada** .
- » Haz una consulta que muestre la vista creada y realiza una captura de pantalla que muestre en su conjunto la vista materializada creada, con la consulta ejecutada y los bits procesados.

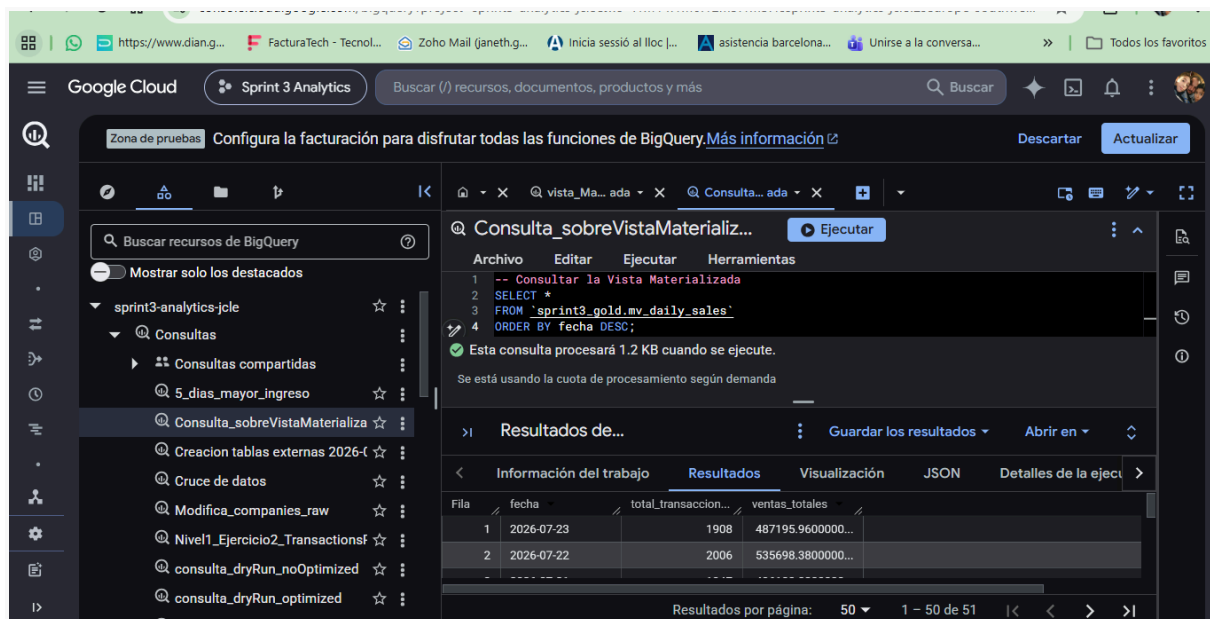
Código generado para crear la Vista:



Link consulta vista materializada:

https://console.cloud.google.com/bigquery?ws=!1m5!1m4!4m3!1ssprint3-analytics-jcle!2ssprint3_gold!3smv_daily_sales

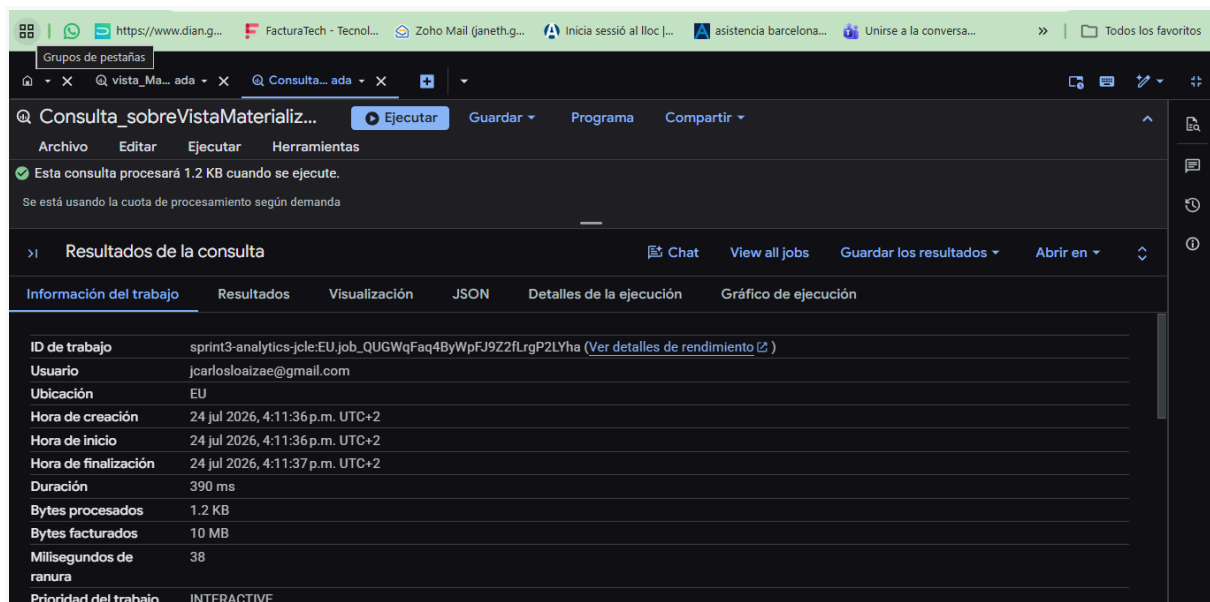
Consulta sobre la vista materializada:



Link de la consulta a la vista materializada:

<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest!3s40538862-1be2-4a8b-bf20-58b9cd4ba9d2!2e1>

captura con información de los Bytes procesados:



La implementación de la Vista Materializada evita recalcular la agregación (SUM(amount) y COUNT(*)) sobre la tabla de hechos en cada petición. Al consultar la vista materializada, BigQuery consume únicamente 1.2 KB de lectura desde la

memoria caché, garantizando un tiempo de respuesta de subsegundos y eliminando costes recurrentes de procesamiento para el cuadro de mando de dirección.

Nivel 2

Nivel 2: SQL Analítico Avanzado

Escenario:

La dirección comienza a hacer preguntas que no se resuelven con un simple `GROUP BY`. Necesitamos análisis de tendencias y comportamiento de usuario.

Ejercicio 1: Perfilado de Clientes VIP (Métricas Agregadas con CTEs)

Escenario:

El equipo de Marketing quiere analizar el comportamiento de nuestros mejores clientes para diseñar la estrategia del próximo año. Definimos "VIP" como aquellos con un gasto acumulado superior a 500 €.

Necesitan un informe que, para cada VIP, muestre su nombre, contacto y su patrón de compra: cuántas veces ha comprado, cuánto gasta de media y cuál fue su compra récord.

Tarea:

» **1. Crea una CTE llamada `VIP_Stats` que agrupe por usuario y calcule :**

- El Gasto Total (SUM).
- La Cantidad de Transacciones (COUNT).
- El Ticket Medio (AVG), redondeado a 2 decimales.
- La Compra Máxima (MAX).
- Filtro: Mantiene sólo aquellos cuyo Gasto Total sea > 500.

» **Crea la CTE con `users_combined` para obtener los datos personales.**

Requisitos de Salida:

- Columnas: `user_id`, `nombre_completo`, `email`, `num_compras`, `ticket_medio`, `max_compra`, `total_gastat`.
- Ordenado por `total_gastado` descendente.

captura del CTE con los cálculos solicitados:

Google Cloud Sprint 3 Analytics

Zona de pruebas Configura la facturación para disfrutar todas las funciones de BigQuery. Más información

Descartar Actualizar

Resultados de la consulta

Visualización JSON Detalles de la ejecución Gráfico de ejecución

Fila	user_id	nombre_completo	email	num_compras	ticket_medio	max_compra	total_gastado
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	94	441.35	554.71	41486.54
2	318	Bnyr Astuw	bnyr.astuw@example.com	91	443.39	615.51	40348.5
3	455	Fwll Phylgmhnd	fwll.phylgmhnd@example.com	66	522.6	724.44	34491.27
4	417	Fqatil Rdwcjvoa	fqatil.rdwcjvoa@example.com	78	410.46	675.78	32016.11
5	185	Molly Gilliam	donec@outlook.couk	110	275.75	659.31	30333.03
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05
8	454	Sfzzoh Xgvfridxs	sfzzoh.xgvfridxs@example.com	81	300.67	604.9	24353.91
9	349	Yxxcjk Fnceqbixmt	yxxcjk.fnceqbixmt@example.c...	53	443.48	675.05	23504.47
10	317	Qikevc Nvfciqdh	qikevc.nvfciqdh@example.com	54	351.69	600.77	18991.45

Resultados por página: 50 1 - 50 de 5000

link del código para la consulta del CTE:

<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest!13sc85effef-887b-45af-a5a0-775df8157360!2e1>

Ejercicio 2: Análisis de Tendencias (Window Functions sobre Vistas)

Escenario:

La Dirección Financiera quiere monitorear la "velocidad de ventas" diaria. Necesitan un informe que compare el rendimiento de cada día contra el día anterior para detectar caídas bruscas o picos de crecimiento (Day-over-Day Growth).



Estrategia de Optimización:

Para este cálculo NO debes ir a la tabla de hechos original ni recalcular sumas. Como buen Data Engineer, debes utilizar la Vista Materializada `sprint3_gold.mv_daily_sales` que creaste en el ejercicio anterior. Esto garantiza que el análisis de tendencias sea instantáneo y no consuma recursos de cómputo innecesarios.



Tarea:

Crea una consulta sobre la vista materializada que genere un informe con las siguientes 4 columnas:

- » 1. **Fecha** : La fecha de venta.
- » 2. **Ventas_Hoy** : El total vendido aquel día.

- » 3. **Ventas_Ayer** : El total vendido el día anterior (usando funciones de ventana).
- » 4. **Diff_Percentual** : El porcentaje de crecimiento o decrecimiento respecto al día anterior, redondeado a 2 decimales.



Pista Técnica:

Utiliza la función `LAG (campo) OVER (ORDER BY data)` para leer el valor de la fila anterior sin necesidad de realizar `JOIN`s complejos.

Captura de código para la consulta de la tendencia de ventas:

```
1 -- SPRINT 4 - NIVEL 2: EJERCICIO 2 (ANÁLISIS DE TENDENCIAS)
2
3 WITH Daily_Sales AS (
4   SELECT
5     fecha,
6     ventas_totales AS ventas_hoy,
7     -- Obtenemos las ventas del día anterior según el orden cronológico
8     LAG(ventas_totales) OVER (ORDER BY fecha ASC) AS ventas_ayer
9   FROM `sprint3_gold_mv_daily_sales`
10 )
11
12 SELECT
13   fecha,
14   ROUND(ventas_hoy, 2) AS ventas_hoy,
15   ROUND(ventas_ayer, 2) AS ventas_ayer,
16   -- Cálculo de variación porcentual: ((Hoy - Ayer) / Ayer) * 100
17   ROUND(
18     SAFE_DIVIDE((ventas_hoy - ventas_ayer), ventas_ayer) * 100,
19     2
20   ) AS diff_percentual
21 FROM Daily_Sales
22 ORDER BY fecha DESC;
```

Se completó la consulta

Resultados de la consulta

Link código consulta de tendencia de ventas:

<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest1!3s8511d786-8e6f-4d57-8673-bc5ce717e2fa!2e1>

Captura de la Vista materializada con las instrucciones solicitadas:

The screenshot shows the Google Cloud BigQuery console. On the left, a sidebar lists various queries, with 'Daily_Sales' selected. The main panel displays the results of a query. The query is 'Daily_Sales' and it has been executed successfully. The results are shown in a table with 5 columns: 'Fila', 'fecha', 'ventas_hoy', 'ventas_ayer', and 'diff_percentual'. The table contains 6 rows of data.

Fila	fecha	ventas_hoy	ventas_ayer	diff_percentual
1	2026-07-23	487195.96	535698.38	-9.05
2	2026-07-22	535698.38	496103.02	7.98
3	2026-07-21	496103.02	514993.71	-3.67
4	2026-07-20	514993.71	512424.71	0.5
5	2026-07-19	512424.71	505195.44	1.43
6	2026-07-18	505195.44	489635.65	3.18

Ejercicio 3: Totales Acumulados (Running Totales sobre Vistas)

Escenario:

El Director Financiero (CFO) necesita visualizar la curva de crecimiento anual. No le interesa tanto lo que se vendió hoy, sino cuánto llevamos facturado en total desde el inicio del año hasta la fecha (YTD - Year To Date).

Tarea:

Utilizando la vista materializada `mv_daily_sales` (para no recalcularse agregaciones base), genera un informe con tres columnas:

- » 1. **Fecha** .
- » 2. **Ventas del Día** : Redondeado a 2 decimales.
- » 3. **Ventas Acumuladas YTD** : Una suma progresiva de las ventas que debe reiniciarse cada **1 de Enero** . El resultado final debe estar redondeado a 2 decimales.



Pista Técnica:

Para conseguir el reinicio anual, necesitas dividir la ventana usando `PARTITION BY EXTRACT(YEAR FROM data)`. La estructura completa sería: `SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`.

Captura de código para visualizar ventas acumuladas:

The screenshot shows the Google Cloud BigQuery console interface. On the left, there's a sidebar with a search bar and a list of datasets including 'creacion_users_combined', 'product_sales_ranking', 'sprint4_eje2_paso1_tablaInterm', 'sprint4_eje2_paso2_tablaOptim', 'sprint4_ejercicio1_diagnostico', 'totalesAcumulados', 'usersVip>500', 'v_marketing_kpis', and 'vista_Materializada'. The main area displays the SQL query for 'totalesAcumulados' in a dark-themed editor. The query is a window function that calculates cumulative sales by date, partitioned by year and ordered by date. Below the query, a status message indicates that the query will process 816 B of data.

```
1 -- SPRINT 4 - NIVEL 2: EJERCICIO 3 (TOTALES ACUMULADOS)
2
3
4 SELECT
5   fecha,
6   ROUND(ventas_totales, 2) AS ventas_del_dia,
7   -- Suma acumulada que se reinicia cada 1 de enero
8   ROUND(
9     SUM(ventas_totales) OVER (
10      PARTITION BY EXTRACT(YEAR FROM fecha)
11      ORDER BY fecha ASC
12      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
13    )
14    , 2)
15   ) AS ventas_acumuladas_ytd
16 FROM `sprint3_gold_mv_daily_sales`
17 ORDER BY fecha ASC;
```

Link código de la consulta ventas acumuladas:

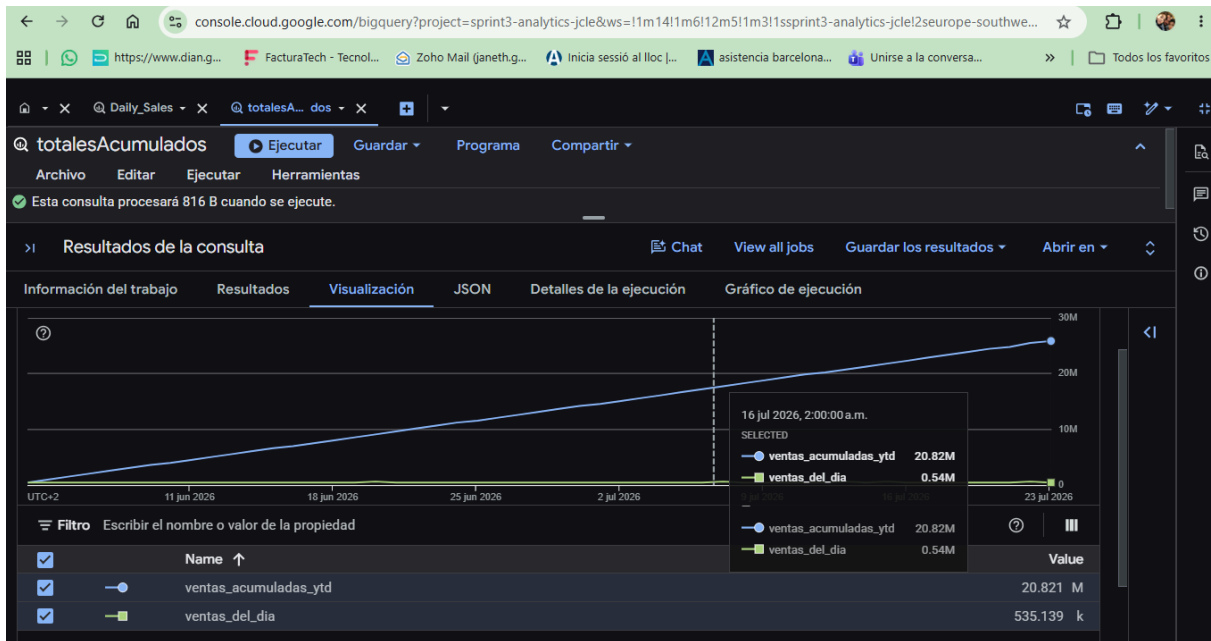
<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest1!3sece94f4e-afe4-4e07-bbea-fe8b7b1bb659!2e1>

Captura con la Información del acumulado de las ventas:

The screenshot shows the Google Cloud BigQuery console with the results of the 'totalesAcumulados' query displayed in a table. The table has four columns: 'Fila', 'fecha', 'ventas_del_dia', and 'ventas_acumuladas_ytd'. The results show a cumulative sales trend from June 3rd to June 8th, 2026. The status message above the table indicates that the query will process 816 B of data.

Fila	fecha	ventas_del_dia	ventas_acumuladas_ytd
1	2026-06-03	502995.09	502995.09
2	2026-06-04	511982.03	1014977.12
3	2026-06-05	507663.39	1522640.51
4	2026-06-06	499310.32	2021950.83
5	2026-06-07	508257.49	2530208.32
6	2026-06-08	502919.35	3033127.67

Captura con visualización del comportamiento de las ventas:



Ejercicio 4: Fidelización y Valor del Cliente (Filtraje Avanzado)

Escenario:

El equipo de Marketing lanza la campaña "A la tercera va la vencida". Quieren enviar un regalo a los usuarios que hayan completado su tercera transacción. Sin embargo, el presupuesto del regalo no es fijo: depende de la "calidad" del cliente en sus inicios. Necesitan saber el Ticket Medio de estas 3 primeras compras para segmentar si envían un detalle básico o un regalo premium.

Tarea:

Genera un listado de los usuarios que han llegado (o superado) su tercera compra, mostrando:

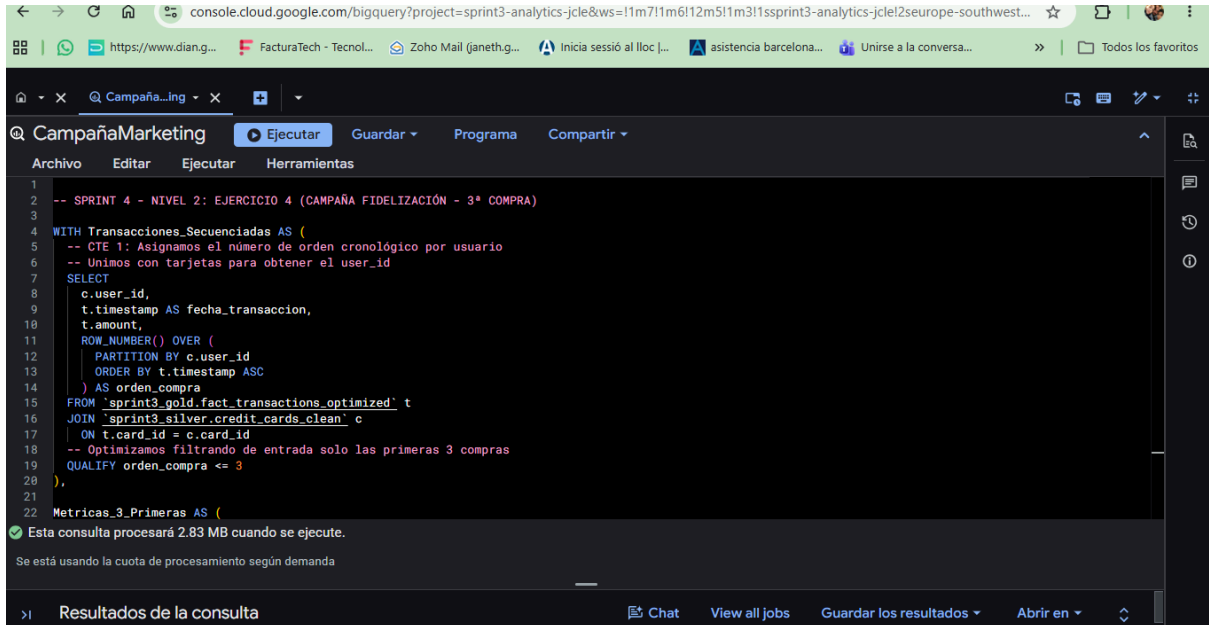
- » 1. **Datos de usuario (user_id, nombre_completo, email)** .
- » 2. **La fecha y el importe exacto de la 3ª compra** .
- » 3. **Media de las 3 primeras** : La media de gasto de sus transacciones 1, 2 y 3.

Pista Técnica:

- » **Debes utilizar la función ROW_NUMBER() para establecer el orden cronológico de las compras** .
- » **Tienes que utilizar la cláusula QUALIFY para filtrar** .
- » **Reto de Optimización** : Busca la estrategia de menor coste computacional. Ten en cuenta que calcular medias sobre todo el

historial histórico es costoso. Trate de filtrar las filas necesarias antes de realizar la agregación final.

Captura código para campaña de marketing:

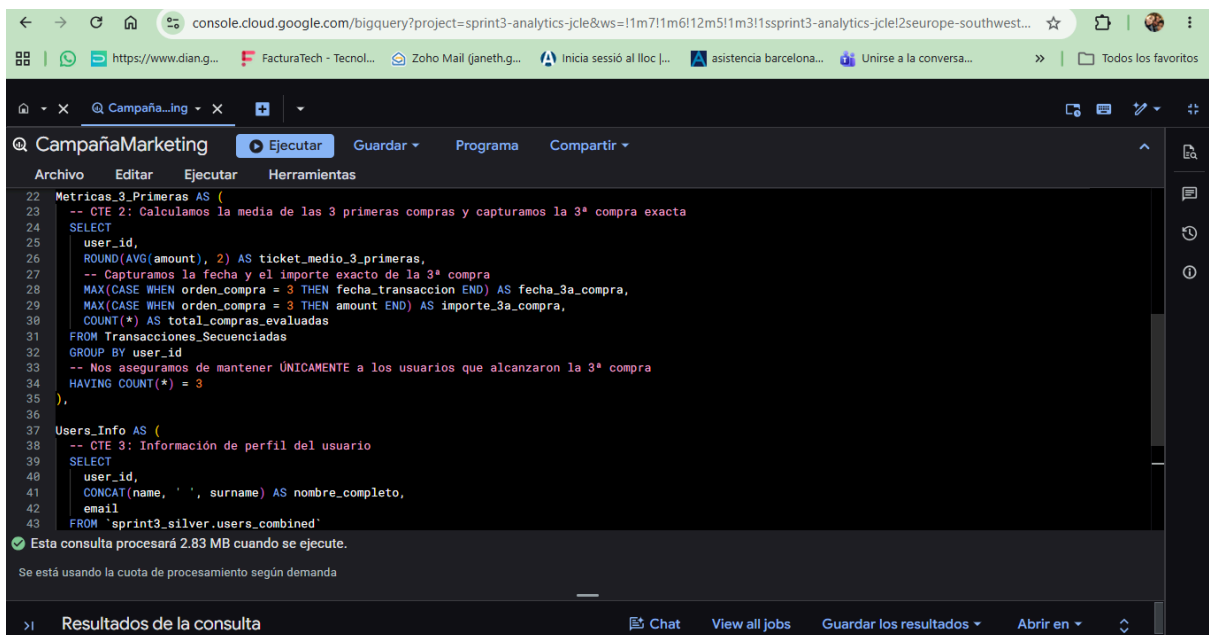


The screenshot shows the Google Cloud BigQuery console interface. The query editor is titled "CampañaMarketing" and contains a SQL query. The query is a CTE named "Transacciones_Secuenciadas" that selects user_id, timestamp, and amount from a table named "sprint3_gold.fact_transactions_optimized". It uses a window function ROW_NUMBER() to rank transactions by timestamp for each user. The query is then joined with "sprint3_silver.credit_cards_clean" on card_id. The final query filters for the first 3 transactions per user (orden_compra <= 3) and calculates metrics for the first 3 transactions.

```
1
2 -- SPRINT 4 - NIVEL 2: EJERCICIO 4 (CAMPAÑA FIDELIZACIÓN - 3ª COMPRA)
3
4 WITH Transacciones_Secuenciadas AS (
5   -- CTE 1: Asignamos el número de orden cronológico por usuario
6   -- Unimos con tarjetas para obtener el user_id
7   SELECT
8     c.user_id,
9     t.timestamp AS fecha_transaccion,
10    t.amount,
11    ROW_NUMBER() OVER (
12      PARTITION BY c.user_id
13      ORDER BY t.timestamp ASC
14    ) AS orden_compra
15  FROM `sprint3_gold.fact_transactions_optimized` t
16  JOIN `sprint3_silver.credit_cards_clean` c
17    ON t.card_id = c.card_id
18  -- Optimizamos filtrando de entrada solo las primeras 3 compras
19  QUALIFY orden_compra <= 3
20 ),
21
22 Metricas_3_Primeras AS (
23   -- CTE 2: Calculamos la media de las 3 primeras compras y capturamos la 3ª compra exacta
24   SELECT
25     user_id,
26     ROUND(AVG(amount), 2) AS ticket_medio_3_primeras,
27     -- Capturamos la fecha y el importe exacto de la 3ª compra
28     MAX(CASE WHEN orden_compra = 3 THEN fecha_transaccion END) AS fecha_3a_compra,
29     MAX(CASE WHEN orden_compra = 3 THEN amount END) AS importe_3a_compra,
30     COUNT(*) AS total_compras_evaluadas
31  FROM Transacciones_Secuenciadas
32  GROUP BY user_id
33  -- Nos aseguramos de mantener ÚNICAMENTE a los usuarios que alcanzaron la 3ª compra
34  HAVING COUNT(*) = 3
35 ),
36
37 Users_Info AS (
38   -- CTE 3: Información de perfil del usuario
39   SELECT
40     user_id,
41     CONCAT(name, ' ', surname) AS nombre_completo,
42     email
43  FROM `sprint3_silver.users_combined`
```

Esta consulta procesará 2.83 MB cuando se ejecute.
Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

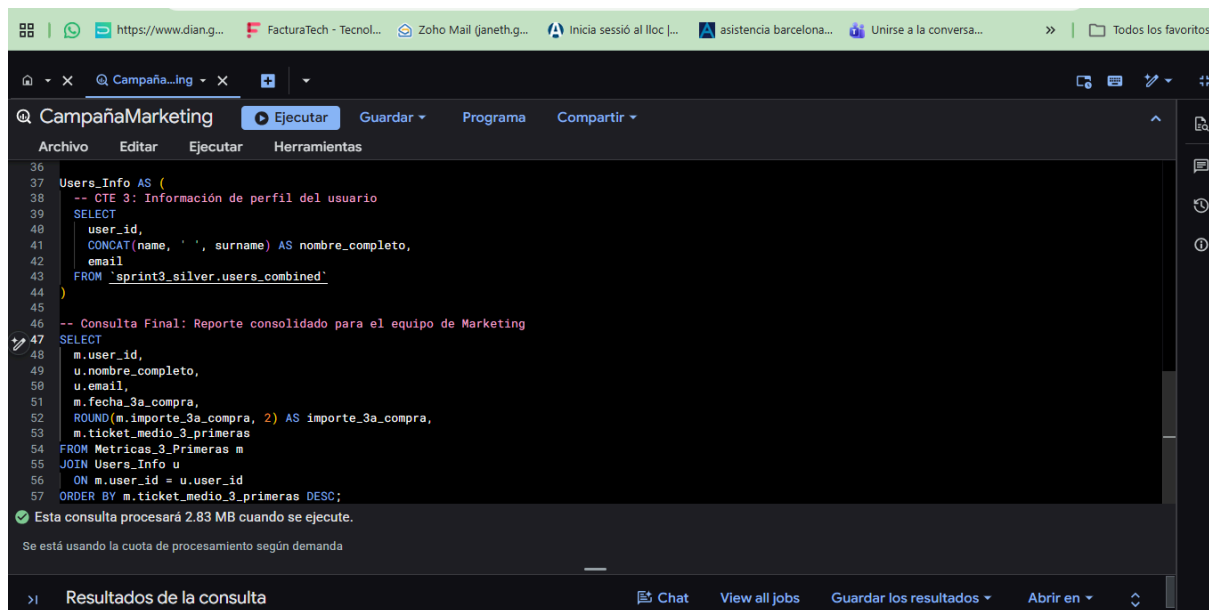


The screenshot shows the Google Cloud BigQuery console interface. The query editor is titled "CampañaMarketing" and contains a SQL query. The query is a CTE named "Metricas_3_Primeras" that calculates the average amount of the first 3 transactions and captures the exact date and amount of the 3rd transaction. It is then joined with "Users_Info" to get user details. The final query filters for users who have made at least 3 transactions.

```
22 Metricas_3_Primeras AS (
23   -- CTE 2: Calculamos la media de las 3 primeras compras y capturamos la 3ª compra exacta
24   SELECT
25     user_id,
26     ROUND(AVG(amount), 2) AS ticket_medio_3_primeras,
27     -- Capturamos la fecha y el importe exacto de la 3ª compra
28     MAX(CASE WHEN orden_compra = 3 THEN fecha_transaccion END) AS fecha_3a_compra,
29     MAX(CASE WHEN orden_compra = 3 THEN amount END) AS importe_3a_compra,
30     COUNT(*) AS total_compras_evaluadas
31  FROM Transacciones_Secuenciadas
32  GROUP BY user_id
33  -- Nos aseguramos de mantener ÚNICAMENTE a los usuarios que alcanzaron la 3ª compra
34  HAVING COUNT(*) = 3
35 ),
36
37 Users_Info AS (
38   -- CTE 3: Información de perfil del usuario
39   SELECT
40     user_id,
41     CONCAT(name, ' ', surname) AS nombre_completo,
42     email
43  FROM `sprint3_silver.users_combined`
```

Esta consulta procesará 2.83 MB cuando se ejecute.
Se está usando la cuota de procesamiento según demanda

Resultados de la consulta



The screenshot shows a Google Cloud BigQuery console window. The title bar indicates the project is 'CampañaMarketing'. The query editor displays a SQL script with the following content:

```
36 Users_Info AS (  
37 -- CTE 3: Información de perfil del usuario  
38 SELECT  
39   user_id,  
40   CONCAT(name, ' ', surname) AS nombre_completo,  
41   email  
42 FROM 'sprint3_silver.users_combined'  
43 )  
44  
45  
46 -- Consulta Final: Reporte consolidado para el equipo de Marketing  
47 SELECT  
48   m.user_id,  
49   u.nombre_completo,  
50   u.email,  
51   m.fecha_3a_compra,  
52   ROUND(m.importe_3a_compra, 2) AS importe_3a_compra,  
53   m.ticket_medio_3_primeras  
54 FROM Metricas_3_Primeras m  
55 JOIN Users_Info u  
56   ON m.user_id = u.user_id  
57 ORDER BY m.ticket_medio_3_primeras DESC;
```

Below the query, a status message indicates: 'Esta consulta procesará 2.83 MB cuando se ejecute.' and 'Se está usando la cuota de procesamiento según demanda'.

The bottom of the window shows the 'Resultados de la consulta' (Query Results) section, which is currently empty.

Link código campaña marketing:

<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest1!3sbf9e9040-f1cb-46d8-ad01-18e0eb2de688!2e1>

Para reducir el consumo de recursos en BigQuery y acelerar el tiempo de respuesta, la consulta se diseñó bajo tres pilares de optimización:

- Filtro temprano con QUALIFY: Al acotar la ventana a orden_compra <= 3 desde el inicio, después de la función de ventana ROW_NUMBER(), descartamos más del 90% del historial de compras que no necesitábamos analizar. Esto evita mover millones de filas innecesarias a través de la memoria del clúster.
- Sin cruces repetidos (Self-JOINS): En lugar de unir la tabla de transacciones consigo misma para localizar la tercera compra (lo que genera operaciones Shuffle muy pesadas entre nodos), usamos agregación condicional (MAX(CASE WHEN...)). Con esto obtenemos la fecha y el importe exacto de la 3ª transacción en una sola lectura de datos.
- Agregación y JOIN final reducidos: Calculamos el ticket promedio sobre un máximo de 3 filas por cliente y filtramos con HAVING COUNT(*) = 3 antes de cruzar con la tabla de usuarios. Así, el JOIN final solo procesa a los clientes que realmente aplican a la campaña.

Nivel 3

Nivel 3: Analytics Engineering (Arrays & Automatización)

Escenario:

Marketing necesita urgentemente un Ranking de Productos más Vendidos.

Ejercicio 1: Desanimiento y Allanamiento de Datos (Unnesting)

Escenario:

Actualmente, en la tabla de transacciones, los productos vendidos están "escondidos" dentro de un Array (una lista de IDs). El equipo de ventas quiere analizar qué productos específicos se venden más, pero no pueden hacerlo si los datos están comprimidos en una lista. Necesitan una Tabla Detallada (Flat Table) donde cada fila represente un producto vendido, aunque esto signifique que el ID de la transacción se repita.

🧩 Tarea:

Crea la mesa `dim_transactions_flat` desnormalizando la información. Tienes que "explotar" el array de productos y cruzarlo con el catálogo maestro para obtener los nombres y precios individuales.

⚙️ Pasos Técnicos:

» 1. Utiliza `CROSS JOIN UNNEST(product_ids)` para transformar el Array en filas individuales.

» 2. Realiza uno `JOIN` con la tabla `products` para obtener el nombre (`name`) y precio (`price`) de cada ítem.

📌 *Nota: Ten cuidado con los tipos de datos al hacer el `JOIN` (Array de `INT64` vs `Product_ID STRING`).*

» 3. No agrupe el resultado. Queremos ver el desglose línea por línea.

✅ Resultado Esperado (Visualización):

Tu mesa final debe verse "plana", donde una transacción con 3 productos ocupa 3 filas:

transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_A	Portátil Básico	99.99
TXN1001	2025-12-08	155.97	SKU_B	Ratón	12.99
TXN1001	2025-12-08	155.97	SKU_C	Funda	29.99
TXN1002	2025-12-07	45.50	SKU_D	Teclado	45.50

👉 Entrega:

- » Una captura de la Vista Previa de tu tabla en la que se vea el DDL ejecutada correctamente.
- » Una captura de pantalla donde se vea el esquema de la nueva tabla.

Captura creación tabla:

The screenshot shows the Google Cloud BigQuery console interface. On the left, there's a sidebar with a search bar and a list of queries. The main area displays a SQL query that has been executed successfully. The query is a DDL statement to create or replace a table named 'dim_transactions_flat' in the 'sprint3_gold' dataset. The query is as follows:

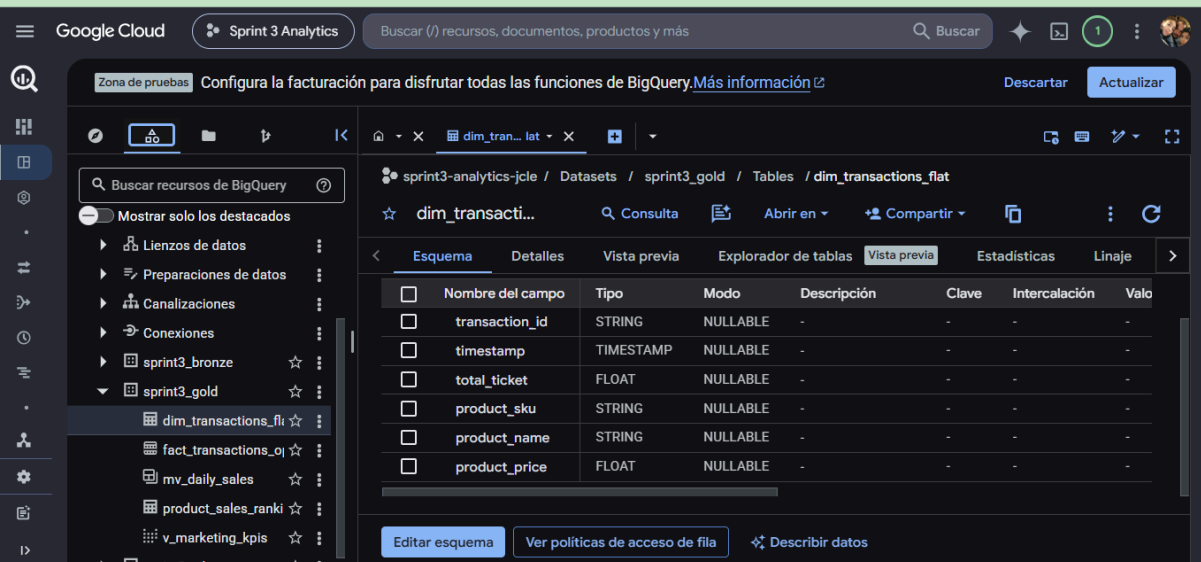
```
CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
SELECT
  t.transaction_id,
```

Below the query, a green checkmark indicates that the query was executed successfully. A message states: "Esta consulta procesará 7.09 MB cuando se ejecute. Se está usando la cuota de procesamiento según demanda". Below this, there's a section titled "Resultados de..." which shows a message: "Esta declaración creó una nueva tabla con el nombre dim_transactions_flat." and a button labeled "Ir a la tabla".

Link código creación tabla:

<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest!13s5e1feae6-f1d4-44a2-9f0d-a2bac7ddf752!2e1>

Captura del esquema de la tabla:



The screenshot shows the Google Cloud BigQuery console interface. On the left, there's a sidebar with navigation options like 'Zona de pruebas', 'Buscar recursos de BigQuery', and a list of datasets including 'sprint3_bronze', 'sprint3_gold', and 'dim_transactions_flat'. The main panel displays the schema for the table 'dim_transactions_flat' under the dataset 'sprint3_gold'. The schema is shown in a table with columns: 'Nombre del campo', 'Tipo', 'Modo', 'Descripción', 'Clave', 'Intercalación', and 'Valo'. The table lists fields: 'transaction_id' (STRING, NULLABLE), 'timestamp' (TIMESTAMP, NULLABLE), 'total_ticket' (FLOAT, NULLABLE), 'product_sku' (STRING, NULLABLE), 'product_name' (STRING, NULLABLE), and 'product_price' (FLOAT, NULLABLE). Below the schema table, there are buttons for 'Editar esquema', 'Ver políticas de acceso de fila', and 'Describir datos'.

Nombre del campo	Tipo	Modo	Descripción	Clave	Intercalación	Valo
transaction_id	STRING	NULLABLE	-	-	-	-
timestamp	TIMESTAMP	NULLABLE	-	-	-	-
total_ticket	FLOAT	NULLABLE	-	-	-	-
product_sku	STRING	NULLABLE	-	-	-	-
product_name	STRING	NULLABLE	-	-	-	-
product_price	FLOAT	NULLABLE	-	-	-	-

Ejercicio 2: El Ranking de Ventas (Agregación Simple)

Escenario:

Ahora que has preparado la infraestructura y dispones de la mesa `dim_transactions_flat` (donde cada fila equivale a un producto vendido), generar informes analíticos es trivial. Ya no necesitas funciones complejas de desapego para responder a preguntas básicas de negocio.

Tarea:

Genera el **Top 5 de productos** más vendidos en la historia de la compañía.

Método:

Consulta directamente tu nueva tabla desnormalizada (`dim_transactions_flat`).

» Al ser la tabla plana, simplemente necesitas una agrupación estándar (`GROUP BY`) para el nombre del producto y una cuenta (`COUNT`) de las filas.

Entrega:

- » El código SQL que genera el ranking.
- » Una captura del resultado con los 5 productos ganadores y cuántas unidades se han vendido de cada uno.

Captura código para ranking de ventas Top 5:

```

1 -- SPRINT 4 - NIVEL 3: EJERCICIO 2 (RANKING DE VENTAS TOP 5)
2
3 SELECT
4   product_sku,
5   product_name,
6   COUNT(*) AS unidades_vendidas,
7   ROUND(SUM(product_price), 2) AS facturacion_total
8 FROM `sprint3_gold.dim_transactions_flat`
9 GROUP BY
10  product_sku,
11  product_name
12 ORDER BY
13  unidades_vendidas DESC
14 LIMIT 5;
  
```

Esta consulta procesará 6.94 MB cuando se ejecute.
Se está usando la cuota de procesamiento según demanda

Link código ranking de ventas Top 5:

<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest1!3s18354239-ce8e-4546-9cef-78818568d781!2e1>

Resultado de la consulta generada:

Fila	product_sku	product_name	unidades_vendidas	facturacion_total
1	52	riverlands the duel	2654	83654.08
2	29	Tully maester Taryl	2635	440572.0
3	21	duel Direwolf	2609	252812.1
4	16	the duel warden	2608	471813.28
5	66	mustafar jinn	2601	5514.12

Resultados por página: 50 1 - 5 de 5

Ejercicio 3: Automatización del Pipeline y Visualización

Escenario:

El informe estático del ejercicio anterior ha gustado mucho, pero el Director de Marketing necesita profesionalizarlo para la toma de decisiones diaria. Tiene dos requerimientos urgentes:

» **1. Lógica Centralizada** : Quieren ver el precio con IVA (21%) desglosado. Sin embargo, dado que el impuesto puede cambiar por país o año, te prohíben poner el cálculo $\text{precio} * 1,21$ directamente en el código ("Hardcoded"). Tienes que utilizar una función reutilizable.

» **2. Frescor de Datos** : El tablero debe estar actualizado automáticamente cada mañana a las 07:00 AM, antes de la reunión diaria de ventas.

🧩 Tarea:

Tienes que evolucionar tu tabla `dim_transactions_flat` para incluir el cálculo de impuestos y automatizar su regeneración.

⚙️ Pasos Técnicos:

» **1. User Defined Functions (UDF)** : Crea una función SQL persistente llamada `calculate_tax(amount)` que reciba un valor numérico y devuelva el resultado aplicando el 21% de impuesto.

» **2. Integración y Orquestación** :

- Modifica el código de creación de la tabla (del Ejercicio 1) para que utilice tu nueva función `calculate_tax` y genere una nueva columna: `product_price_tax_inc`.
- Configura una Scheduled Query en BigQuery para que esta sentencia (CTAS) se ejecute automáticamente todos los días a las 07:00 AM.

» **3. Visualización (BI)** : Conecta Looker Studio a tu mesa `dim_transactions_flat` y crea un Dashboard: "Monitor de Rendimiento de Ventas"

👉 Entrega:

- » El código SQL de la UDF.
- » El código SQL actualizado de la creación de la tabla.
- » Una captura de pantalla demostrando que la Scheduled Query está programada.
- » Un vínculo del gráfico a Looker Studio.

📌 Propuesta de Dashboard:

"Monitor de Rendimiento de Ventas"

Configura una página con 3 secciones clave:

» 1. Los "Big Numbers" (KPIs Generales) :

En la parte superior, coloca tres tarjetas de resultados (Scorecards) para dar contexto inmediato.

- **KPI 1: Ingresos Totales (Con IVA)**
 - **Métrica** : `SUM(product_price_tax_inc)`
 - *Por qué* : Muestra el volumen real de dinero que entra.
- **Transacciones Únicas (Pedidos)**
 - **Métrica** : `COUNT_DISTINCT(transaction_id)`
- **Transacciones Únicas (Pedidos)**
 - **Campo Calculado** : Crea un campo en Looker: `SUM(product_price_tax_inc) / COUNT_DISTINCT(transaction_id)`

» 2. Evolución Temporal (Gráfico de Series Temporales)

Bajo los KPIs, un gráfico de línea que ocupe todo el ancho.

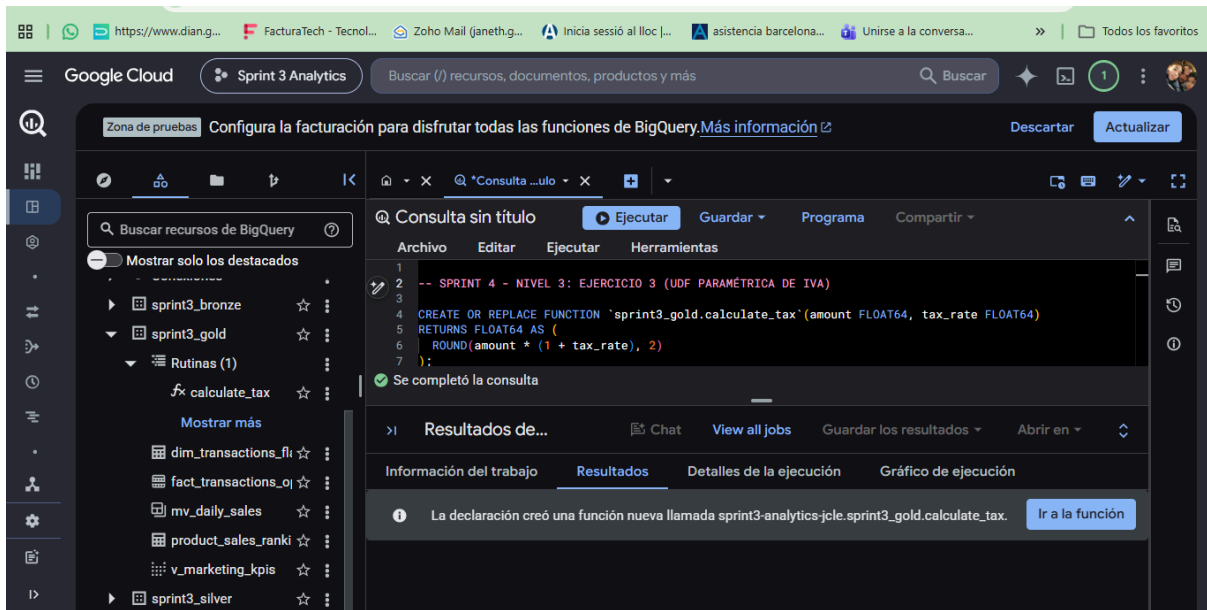
- **Dimensión** : timestamp (Desglosado por Fecha).
- **Métrica** : `SUM(product_price_tax_inc)`.
- **Línea de Referencia (Opcional)** : Añade una media constante o una línea de tendencia.
- **Valor** : Permite a Marketing ver si las ventas suben o bajan día a día, o si hubo un pico por alguna campaña específica.

» El "Top Productes" Enriquecido (Tabla con Mapa de Calor)

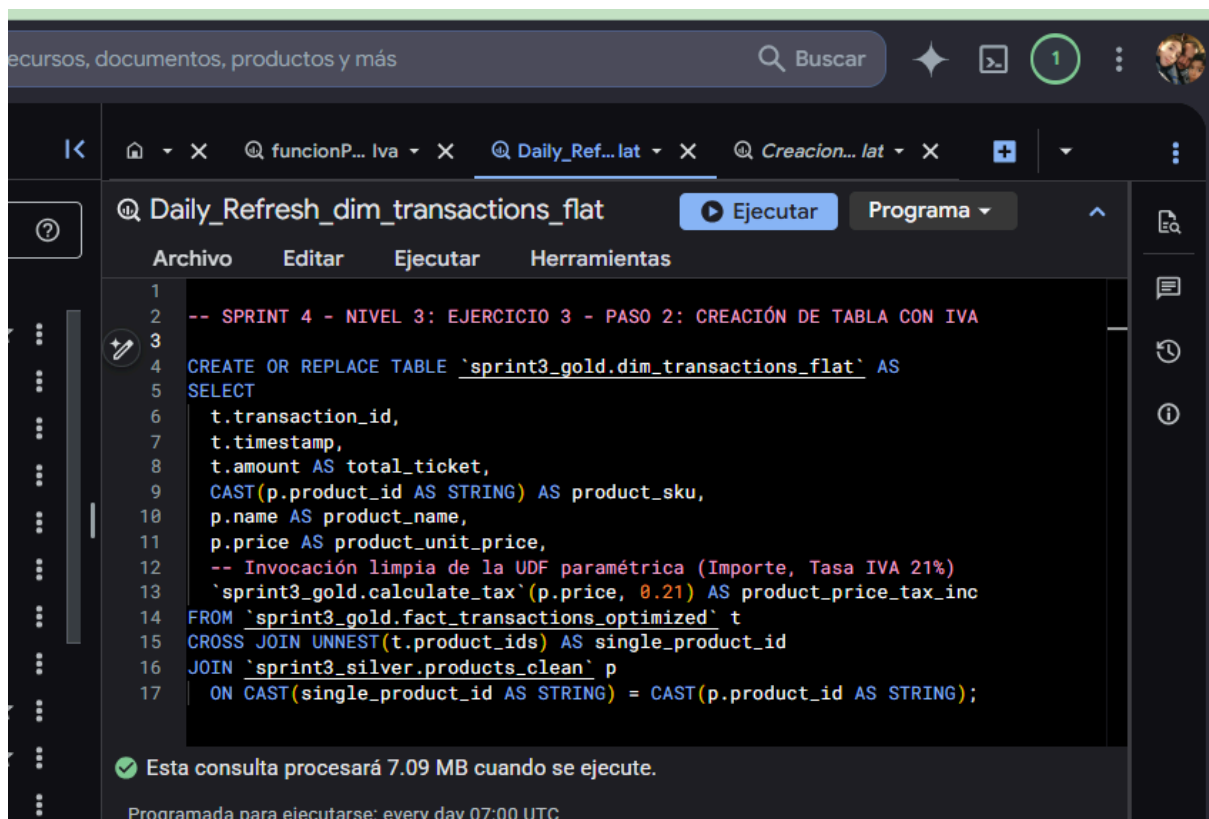
Olvídate del gráfico de barras simple. Utiliza una Tabla con Barras incrustadas o Mapa de Calor.

- **Dimensión** : `product_name`.
- **Métrica 1** : `COUNT(*)` → Renúmbalo en "Unidades Vendidas".
- **Métrica 2** : `SUM(product_price_tax_inc)` → Renúmbalo en "Ingresos Totales".
- **Métrica 3** : `SUM(product_price_tax_inc) - SUM(product_unit_price)` → Renúmbalo en "IVA Recaudado" (Esto demuestra el valor de tu UDF).
- **Estilo** : En la configuración de estilo de la tabla, activa la opción "Muestra número como: Barra" o "Mapa de calor" para la columna de Ingresos.

Captura creacion funcion de la UDF:



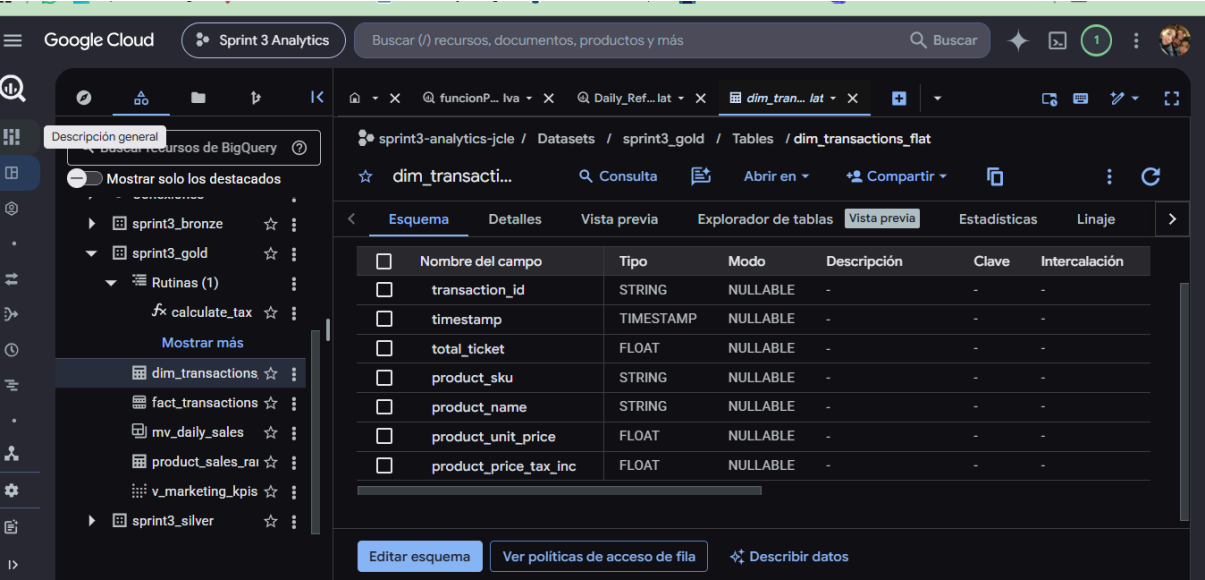
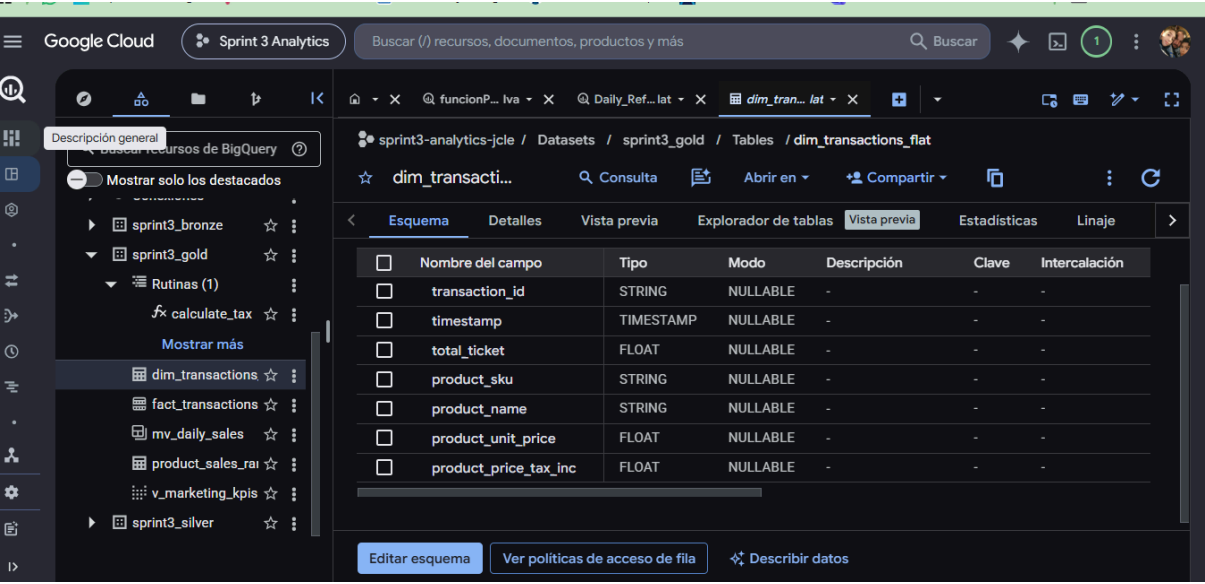
Captura de la evolución de la tabla:



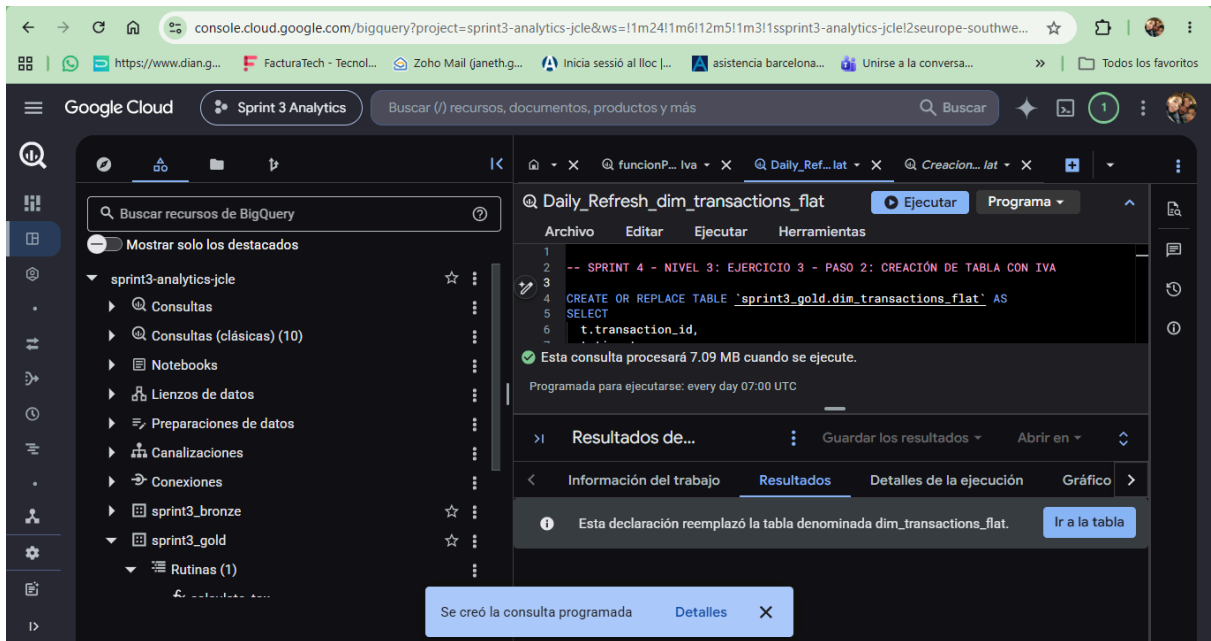
Link código evolución tabla:

<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest1!3s4a6aeb46-c9f9-4d43-a77e-bd193aa1eb92!2e1>

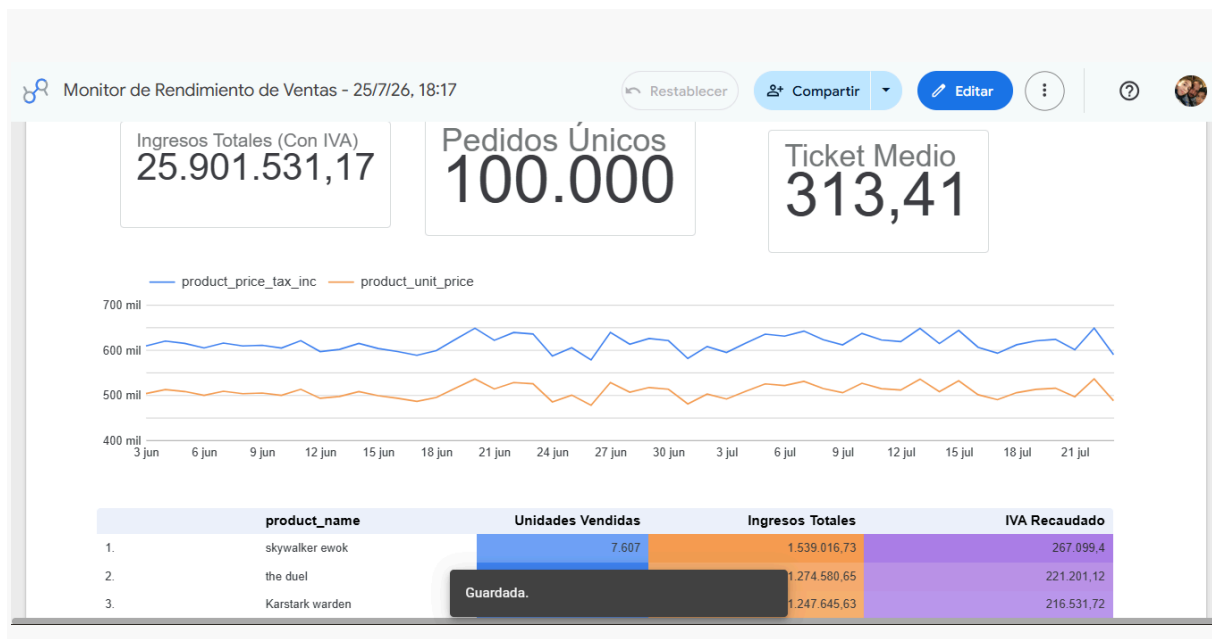
Captura nueva tabla con columna generada:



captura de la programación de la consulta:



Dashboard:



enlace dashboard:

<https://datastudio.google.com/reporting/0ecd6894-bbb3-4c2d-976a-8fd051c3dc4c>